

SOEN 341 – Software Process

CareerConnect

Generative AI Usage Log

Student	Philip Ear
Project	CareerConnect – Job Search and Application Tracking Platform
Sprint	Sprint 1
Primary Work	Backend – Profile and Resume Management
Submission Date	September 28 th 2026

1. Purpose of This Log

This document records my use of generative AI during Sprint 1 of the CareerConnect project. For each meaningful use, I identify the task, why AI was used, the content suggested by AI, how I validated it, whether I accepted, modified, or rejected it, and what I learned from the interaction. The evidence provided under each entry is intended to contain the corresponding ChatGPT conversation link or the complete prompt and relevant response evidence required for the course submission.

2. Summary of AI-Assisted Work

Entry	Task / Purpose	Area	Decision
1	Generate 10 user stories	Sprint planning	Reviewed / Modified
2	Understand existing backend structure	Backend architecture	Accepted
3	Design and implement Profile API	Profile backend	Modified
4	Design resume database storage	Resume backend	Modified
5	Implement resume upload API	Resume backend	Modified
6	Implement resume list/download/delete	Resume backend	Modified
7	Connect routes to Express application	Integration	Accepted
8	Generate commands for backend testing	Testing	Accepted
9	Debug resume endpoint mismatch	Debugging	Accepted
10	Verify frontend/backend integration	Integration testing	Modified

3. Detailed AI Usage Entries

Entry 1 – Generate 10 User Stories

Task ID / Title	Sprint 1 – AI-generated user-story exercise
Purpose	Use generative AI to propose 10 possible CareerConnect user stories as required for the Sprint 1 AI usage exercise.
AI-Suggested Content	AI proposed user stories covering account/profile management, resumes, job search, applications, saved jobs, application status tracking, and recruiter activities.
Validation	I compared the proposed stories with the CareerConnect project requirements and checked that each story described functionality relevant to the platform. The team should keep its independently brainstormed stories/features separate from this AI-generated set.
Decision	Reviewed / Modified
Reflection	AI was useful for quickly producing a starting set of stories, but the stories still needed to be checked against the assignment scope and refined into issues/tasks appropriate for the team.
Responsible Person	Entire team did this section together

Evidence: Appendix A.1 –

Prompt: “Generate me 10 user stories related to the project of CareerConnect. ”

“AI-generated response

US-01 — User Registration (Sprint 1)

As a **job seeker**, I want to create an account so that I can access CareerConnect features.

US-02 — User Login (Sprint 1)

As a **registered user**, I want to log in securely so that I can access my personal account and information.

US-03 — Profile Management (Sprint 1)

As a **job seeker**, I want to create and edit my profile so that my personal and professional information remains up to date.

US-04 — Resume Upload

As a **job seeker**, I want to upload my resume so that I can use it when applying for jobs.

US-05 — Resume Management

As a **job seeker**, I want to view, replace, or delete my uploaded resumes so that I can keep my resume information current.

US-06 — Browse Job Postings

As a **job seeker**, I want to browse available job postings so that I can discover employment opportunities.

US-07 — Search for Jobs

As a **job seeker**, I want to search for jobs using keywords so that I can quickly find relevant opportunities.

US-08 — Filter Job Postings

As a **job seeker**, I want to filter job postings based on criteria such as location or job type so that I can narrow down my search.

US-09 — Save Jobs

As a **job seeker**, I want to save interesting job postings so that I can easily return to them later.

US-10 — Submit Job Application

As a **job seeker**, I want to submit an application to a job posting so that I can apply directly through CareerConnect.”

Entry 2 – Understand Existing Backend Structure

Task ID / Title	Backend preparation – existing Express/SQLite/JWT code
Purpose	Understand how the existing server, database, authentication routes, and JWT middleware worked before adding my profile and resume features.
AI-Suggested Content	AI explained the responsibilities of app.js, server.js, db.js, the authentication routes, and the JWT authentication middleware. It recommended reusing req.user.id from the verified token for protected profile and resume operations.
Validation	I checked the explanation against the actual project files. I confirmed that registration already creates a profile row and that protected routes can identify the signed-in user from the authentication middleware.
Decision	Accepted
Reflection	This helped me understand how to extend the existing backend instead of creating duplicate authentication or user-identification logic.
Responsible Person	Philip Ear

Evidence: Appendix A.2 –

Prompt: I am responsible for the backend Profile and Resume Management section of our CareerConnect project. The project uses Node.js, Express, SQLite, JWT authentication, and a React frontend. Can you briefly explain how I should approach implementing my section and how the different backend parts connect together?

Response

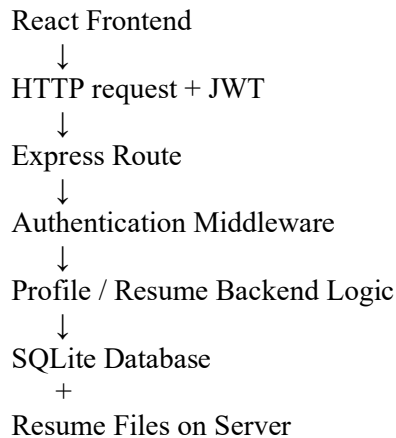
For your section, the backend can be divided into two main features: **Profile Management** and **Resume Management**.

For **Profile Management**, first understand the existing authentication system. When a user logs in, the server gives them a JWT token. Protected profile routes can use this token to determine which user is making the request. You can then create a `GET /api/profile` endpoint to retrieve the logged-in user’s profile and a `PUT /api/profile` endpoint to update fields such as name, headline, location, phone, bio, skills, LinkedIn URL, and company name.

For **Resume Management**, create a database table that stores information about each uploaded resume, including the user ID, original filename, stored filename, file type, file size, and upload date. The actual PDF or DOCX file can be stored in an uploads folder on the server. A library such as **Multer** can handle the file upload.

You can then create protected API endpoints for uploading, listing, downloading, and deleting resumes. Each resume query should use the authenticated user's ID so users can only access their own files.

The overall connection is:



After implementing each endpoint, test the backend independently using commands such as `curl` before connecting it to the React frontend. This makes it easier to determine whether a problem is coming from the backend or the frontend.

Entry 3 – Profile API Design and Implementation

Task ID / Title	Backend Profile feature
Purpose	Create authenticated API endpoints for loading and updating a CareerConnect user profile.
AI-Suggested Content	AI suggested protected GET <code>/api/profile</code> and PUT <code>/api/profile</code> endpoints. The proposed implementation retrieves the profile using <code>req.user.id</code> , converts stored skills to/from JSON, validates required fields, updates profile information, and returns the updated profile.
Validation	I reviewed the code against the existing profile table and authentication middleware. I confirmed that a separate profile creation endpoint was unnecessary because registration already creates the profile. I tested loading and saving profile information with a JWT and later verified persistence through the React frontend.
Decision	Modified
Reflection	The generated structure provided a useful starting point, but I had to adapt field names and responses to the team's existing server and verify that saved information persisted in SQLite.
Responsible Person	Philip Ear

Evidence: Appendix A.3 –

Prompt: I am responsible for the backend Profile feature for our CareerConnect project. We are using Node.js, Express, SQLite, and JWT authentication. Registration already creates a user and profile. How should I implement the backend so that a logged-in user can retrieve and update their profile? Please explain the endpoints and give me a basic implementation.

Response:

Since registration already creates the user's profile, you do not need another endpoint to create one. The Profile backend mainly needs two authenticated endpoints:

GET /api/profile

PUT /api/profile

GET /api/profile retrieves the profile belonging to the currently logged-in user. Instead of allowing the frontend to provide a user ID, use the ID obtained from the verified JWT:

`req.user.id`

The basic flow is:

GET /api/profile

↓

`requireAuth`

↓

`req.user.id`

↓

`SELECT profile FROM SQLite`

↓

Return profile as JSON

For updating the profile, PUT /api/profile receives the editable profile information from the frontend, such as:

```
{
  "fullName": "Test User",
  "headline": "Computer Engineering Student",
  "location": "Montreal, QC",
  "phone": "514-555-1234",
  "bio": "Interested in software development.",
  "skills": ["Java", "C++", "JavaScript"],
  "linkedinUrl": "https://linkedin.com/in/test",
  "companyName": ""
}
```

The backend should validate important fields such as `fullName`, convert the skills array to JSON for SQLite storage, and update the profile associated with `req.user.id`.

A basic route structure would be:

```
const express = require('express');
const { getDb } = require('../db');
const { requireAuth } = require('../middleware/auth');
```

```
const router = express.Router();
```

```
router.get('/', requireAuth, (req, res) => {
  // Find profile using req.user.id
});
```

```
router.put('/', requireAuth, (req, res) => {
  // Validate request
  // Update profile belonging to req.user.id
  // Return updated profile
});
```

```
module.exports = router;
```

Then register the router in `app.js`:

```
app.use('/api/profile', require('./routes/profile'));
```

Using `requireAuth` on both endpoints means the server first verifies the JWT and determines which user is making the request. This prevents the frontend from simply requesting another user's profile by changing a user ID.

Entry 4 – Resume Database Design

Task ID / Title	Backend Resume feature – database
Purpose	Determine how uploaded resumes should be represented in the SQLite database while storing the actual files on disk.
AI-Suggested Content	AI proposed a resumes table containing an ID, user ID foreign key, original filename, stored filename, file type, file size, and upload timestamp. The actual uploaded files would be stored in a server uploads directory.
Validation	I checked that the user_id foreign key could associate each resume with its owner and that resume queries could be restricted by both resume ID and authenticated user ID. I also reviewed which generated files/directories should be excluded from Git.
Decision	Modified
Reflection	Separating file metadata from the physical file made the implementation easier to manage and query, but the schema still needed to match the features selected by the team.
Responsible Person	Philip Ear

Evidence: Appendix A.4 –

Prompt: I am responsible for the backend Resume Management feature for our CareerConnect project. We are using Node.js, Express, SQLite, and JWT authentication. Users need to be able to upload and manage resumes. How should I design the database for this feature? Should the actual resume file be stored in SQLite or should I only store information about the file?

Response: For CareerConnect, a simple approach is to **store the actual resume files on the server** and store information about each file in SQLite.

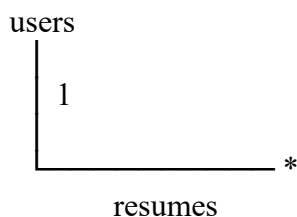
A `resumes` table could be designed as:

```
CREATE TABLE IF NOT EXISTS resumes (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  user_id INTEGER NOT NULL  
    REFERENCES users(id) ON DELETE CASCADE,  
  original_name TEXT NOT NULL,  
  stored_name TEXT NOT NULL,  
  file_type TEXT NOT NULL,  
  file_size INTEGER NOT NULL,  
  uploaded_at TEXT NOT NULL DEFAULT (datetime('now'))  
);
```

Each field has a specific purpose:

- `id` uniquely identifies each resume.
- `user_id` connects the resume to the user who uploaded it.
- `original_name` stores the filename that the user originally uploaded, such as `CV.pdf`.
- `stored_name` stores the filename used internally by the server.
- `file_type` records the type of uploaded file, such as PDF or DOCX.
- `file_size` records the file size and can help with validation/display.
- `uploaded_at` records when the resume was uploaded.

The relationship would be:



One user can therefore have multiple resumes, while every resume belongs to one user.

The actual files can be organized separately:

```

server/
├── src/
│   └── ...
├── uploads/
│   └── resumes/
│       ├── 1-1790535509.pdf
│       └── 1-1790535620.docx

```

The database then stores the information needed to locate the corresponding file.

When retrieving, downloading, or deleting a resume, the backend should check both the resume ID and authenticated user's ID:

```

SELECT *
FROM resumes
WHERE id = ? AND user_id = ?;

```

This helps ensure that a logged-in user can only manage resumes belonging to their own account.

Entry 5 – Resume Upload API

Task ID / Title	Backend Resume feature – upload
Purpose	Implement authenticated PDF/DOCX resume uploads for job seekers.
AI-Suggested Content	AI suggested using Multer with disk storage, a dedicated uploads/resumes directory, generated stored filenames, allowed MIME types for PDF and DOCX, a 5 MB file-size limit, and an authenticated POST /api/resume endpoint that stores metadata in SQLite.
Validation	I installed the required package, reviewed the allowed file types and upload directory, and tested the endpoint using curl with a real PDF. I confirmed that the endpoint requires a valid JWT and that the uploaded resume metadata can be retrieved afterward.
Decision	Modified
Reflection	The generated upload code reduced setup time, but testing was necessary because file paths, endpoint naming, validation behavior, and project conventions can differ from generated examples.
Responsible Person	Philip Ear

Evidence: Appendix A.5 –

Prompt: I have already designed the resume database for the CareerConnect backend. Now I need to implement the resume upload feature using Node.js, Express, SQLite, and JWT authentication. Users should be able to upload PDF or DOCX resumes. How should I implement the upload endpoint, validate the file, save it on the server, and store its information in the database?

Response: For the resume upload feature, you can use **Multer**, an Express middleware designed to handle file uploads.

First, install it:

```
npm install multer
```

Create a resume route that uses JWT authentication before accepting the upload:

```
POST /api/resume
  ↓
requireAuth
  ↓
Multer
  ↓
Validate file
  ↓
Save file in uploads/resumes/
  ↓
Store metadata in SQLite
  ↓
Return uploaded resume information
```

Configure Multer to save uploaded files in a dedicated folder such as:

```
server/uploads/resumes/
```

The upload should accept a single file using the field name:

```
resume
```

For example:

```
router.post(
  '/',
  requireAuth,
  upload.single('resume'),
  (req, res) => {
    // Save resume information to database
  }
);
```

The backend should validate the uploaded file. For CareerConnect, you can restrict uploads to **PDF and DOCX** and apply a file-size limit, such as 5 MB.

After Multer successfully saves the file, store its metadata in the `resumes` table:

```
db.prepare(`
  INSERT INTO resumes
    (user_id, original_name, stored_name, file_type, file_size)
  VALUES (?, ?, ?, ?, ?)
`).run(
  req.user.id,
  req.file.originalname,
  req.file.filename,
  req.file.mimetype,
  req.file.size
);
```

Using `req.user.id` is important because it associates the uploaded resume with the user identified by the JWT rather than trusting a user ID provided by the frontend.

The actual file remains in the server upload directory while SQLite stores the information needed to identify and manage it.

Entry 6 – Resume List, Download, and Delete

Task ID / Title	Backend Resume management
Purpose	Allow users to view their uploaded resumes, download them, and delete them.
AI-Suggested Content	AI proposed authenticated GET <code>/api/resume</code> , GET <code>/api/resume/:id/download</code> , and DELETE <code>/api/resume/:id</code> routes. Each operation checks the authenticated user's ID so one user cannot access another user's resume.
Validation	I reviewed the SQL conditions to ensure they include <code>user_id</code> and tested the resume list through the frontend. The frontend successfully displayed the stored CV, confirming that the list endpoint and authentication were communicating correctly.
Decision	Modified
Reflection	The ownership check was an important part of the generated design. I learned that simply knowing a resume ID should not be sufficient to retrieve or delete the file.
Responsible Person	Philip Ear

Evidence: Appendix A.6 –

Prompt: I already implemented resume uploading for the CareerConnect backend using Node.js, Express, SQLite, Multer, and JWT authentication. Now I need users to manage their uploaded resumes. How should I implement the backend so users can view their resumes, download a resume, and delete a resume while making sure they can only access their own files?

Response: For resume management, you can add three main authenticated endpoints:

```
GET /api/resume
GET /api/resume/:id/download
DELETE /api/resume/:id
```

The first endpoint retrieves all resumes belonging to the logged-in user:

```
router.get('/', requireAuth, (req, res) => {
  const db = getDb();

  const resumes = db.prepare(`
    SELECT id, original_name, file_type, file_size, uploaded_at
    FROM resumes
    WHERE user_id = ?
    ORDER BY uploaded_at DESC
  `).all(req.user.id);

  res.json({ resumes });
});
```

For downloading, the backend should find the requested resume using both the resume ID and the authenticated user's ID:

```
const resume = db.prepare(`
  SELECT *
  FROM resumes
  WHERE id = ? AND user_id = ?
`).get(req.params.id, req.user.id);
```

If the resume exists and belongs to the user, the server can locate the physical file and send it for download. Otherwise, it should return a 404 response.

Deleting works similarly. The backend first verifies ownership, removes the physical file from the uploads directory, and then removes its database record:

```
DELETE /api/resume/:id
↓
JWT authentication
↓
Find resume using id + user_id
↓
Verify resume belongs to user
↓
Delete file from server
```

↓
Delete database record
↓
Return success response

An important security rule is to avoid searching only by:

WHERE id = ?

Instead, use:

WHERE id = ? AND user_id = ?

This prevents a user from changing the resume ID in the URL to access another user's resume.

Entry 7 – Express Route Integration

Task ID / Title	Backend integration
Purpose	Register the new profile and resume routers in the existing Express application.
AI-Suggested Content	AI recommended mounting the profile router at /api/profile and the resume router at /api/resume while preserving the existing authentication route and final 404/error middleware.
Validation	I checked the current app.js route order and verified that the routes are mounted before the catch-all 404 handler. I restarted the server after changes and tested the endpoints.
Decision	Accepted
Reflection	This was a small change, but it showed why route order matters in Express: a feature route placed after the catch-all 404 handler would never be reached.
Responsible Person	Philip Ear

Evidence: Appendix A.7 –

Prompt: I have created the backend Profile and Resume routes for CareerConnect using Node.js and Express. The project already has authentication routes and a main `app.js` file. How should I connect my new Profile and Resume routes to the existing Express application so that the frontend can access them?

Response: In Express, the Profile and Resume route files need to be registered in the main application. This allows requests coming from the frontend to be directed to the correct backend feature.

If the route files are:

server/src/routes/profile.js
server/src/routes/resume.js

they can be registered in `app.js`:

```
app.use('/api/auth', require('./routes/auth'));
app.use('/api/profile', require('./routes/profile'));
app.use('/api/resume', require('./routes/resume'));
```

This creates the following API structure:

```
/api/auth    → Authentication routes
/api/profile  → Profile routes
/api/resume   → Resume routes
```

For example, if `profile.js` contains:

```
router.get('/', requireAuth, ...);
```

mounting it at `/api/profile` makes the complete endpoint:

GET `/api/profile`

Similarly:

```
router.post('/', requireAuth, ...);
```

inside `resume.js` becomes:

POST `/api/resume`

The feature routes should also be registered **before the catch-all 404 middleware**:

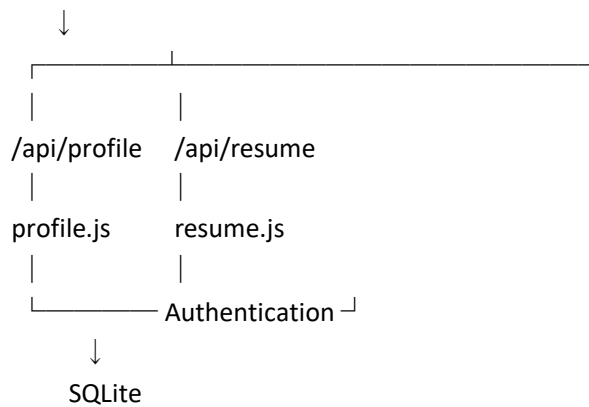
```
app.use('/api/profile', require('./routes/profile'));
app.use('/api/resume', require('./routes/resume'));
```

```
app.use((req, res) => {
  res.status(404).json({ error: 'Not found.' });
});
```

Otherwise, a request could reach the 404 handler before Express has an opportunity to process the feature route.

The overall integration becomes:

```
React Frontend
  ↓
Express app.js
```



Entry 8 – Backend Testing Commands

Task ID / Title	Backend testing
Purpose	Create repeatable terminal commands to test registration/login, authenticated profile requests, and resume upload.
AI-Suggested Content	AI generated curl commands for registration and login, showed how to save the returned JWT in a shell variable, and demonstrated how to send the Bearer token with protected requests and multipart resume uploads.
Validation	I ran the commands locally. Registration/login returned a token, the token authenticated profile requests, and a real CV was used to test the resume endpoint.
Decision	Accepted
Reflection	The commands gave me a quick way to isolate backend behavior before depending on the frontend, which made debugging easier.
Responsible Person	Philip Ear

Evidence: Appendix A.8 –

Prompt: I finished implementing the Profile and Resume backend features for CareerConnect. The backend uses Node.js, Express, SQLite, JWT authentication, and Multer. How can I test my backend from the terminal before relying on the React frontend? Give me commands to test login, save the JWT token, retrieve/update a profile, upload a resume, and retrieve the resume list.

Response: You can use `curl` commands from the terminal to test the API directly. This helps verify that the backend works independently from the frontend.

First, make sure the server is running:

```
cd server
npm run dev
```

Then log in with an existing test account:

```
curl -X POST http://localhost:4000/api/auth/login \
-H "Content-Type: application/json" \
```

```
-d '{"email":"test@example.com","password":"password123"}'
```

The server should return a JWT token. You can save it in a terminal variable:

```
TOKEN=$(curl -s -X POST http://localhost:4000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"test@example.com","password":"password123"}' \
| python3 -c "import sys,json; print(json.load(sys.stdin)['token'])")
```

Verify that the token was saved:

```
echo $TOKEN
```

Then test the authenticated Profile endpoint:

```
curl http://localhost:4000/api/profile \
-H "Authorization: Bearer $TOKEN"
```

You can test updating the profile with:

```
curl -X PUT http://localhost:4000/api/profile \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "fullName":"Test User",
  "headline":"Computer Engineering Student",
  "location":"Montreal, QC",
  "phone":"514-555-1234",
  "bio":"Interested in software development.",
  "skills":["Java","C++","JavaScript"],
  "linkedinUrl":"",
  "companyName":""
}'
```

To test resume uploading, use a real file path on the computer:

```
curl -X POST http://localhost:4000/api/resume \
-H "Authorization: Bearer $TOKEN" \
-F resume=@\$HOME/Desktop/CV.pdf
```

Finally, retrieve the user's uploaded resumes:

```
curl http://localhost:4000/api/resume \
-H "Authorization: Bearer $TOKEN"
```

Testing in this order makes it easier to determine whether an error comes from authentication, the API endpoint, the database, file uploading, or later from the frontend.

Entry 9 – Debug Resume Endpoint Mismatch

Task ID / Title	Debugging – 404 Not Found
Purpose	Determine why a resume upload request returned <code>{"error": "Not found."}</code> .
AI-Suggested Content	AI compared the request URL with the Express route registration and identified that the server was mounted at singular <code>/api/resume</code> while the curl request used plural <code>/api/resumes</code> .
Validation	I checked <code>app.js</code> and confirmed the singular route. I corrected the request URL and continued testing.
Decision	Accepted
Reflection	This interaction demonstrated the value of comparing the actual registered route with the request instead of changing unrelated upload or database code when a 404 occurs.
Responsible Person	Philip Ear

Evidence: Appendix A.9 –

Prompt: I am testing the Resume Upload backend for CareerConnect using `curl`. My Express application has a resume route, but when I try to upload my CV I receive `{"error": "Not found."}`. How can I debug this 404 error and determine whether the problem is with my route, authentication, or upload code?

Response: A `404 Not Found` usually means the request did not match an endpoint registered in Express. Before changing the database or Multer upload code, compare the URL used in the `curl` command with the route registered in `app.js`.

The application registers the Resume router as:

```
app.use('/api/resume', require('./routes/resume'));
```

Therefore, if `resume.js` contains:

```
router.post('/', requireAuth, upload.single('resume'), ...);
```

the complete endpoint is:

POST `/api/resume`

If the test command instead uses:

POST `/api/resumes`

with an extra `s`, Express does not find the route and eventually reaches the application's 404 handler:

```
{
  "error": "Not found."
}
```

The upload command should therefore use:

```
curl -X POST http://localhost:4000/api/resume \
-H "Authorization: Bearer $TOKEN" \
-F resume=@$HOME/Desktop/CV.pdf
```

When debugging an Express 404, check the **HTTP method**, the base path registered in `app.js`, the path inside the router, and whether the server was restarted after backend changes.

Entry 10 – Frontend / Backend Integration Verification

Task ID / Title	Integration testing support
Purpose	Verify that the Profile and Resume frontend screens can authenticate and communicate with my backend endpoints.
AI-Suggested Content	AI explained how to place a development JWT in browser localStorage, refresh the React application, save profile changes, verify persistence, and test the resume list/upload/download/delete flow.
Validation	I loaded the Profile page and confirmed that stored profile information appeared. I also loaded the Resumes page and confirmed that the existing CV was retrieved from the backend. A date-formatting issue was visible in the UI and was identified as a frontend formatting problem rather than a backend retrieval failure.
Decision	Modified
Reflection	Testing the complete flow helped distinguish backend issues from frontend presentation issues. It also confirmed that authentication, API requests, and database retrieval were working together.
Responsible Person	Philip Ear

Evidence: Appendix A.10 –

Prompt: I finished implementing the Profile and Resume backend for CareerConnect, and my teammate has created the React frontend for these features. How can I verify that the frontend is correctly communicating with my backend? The backend uses JWT authentication, Express, SQLite, and Multer, while the frontend runs with Vite.

Response: To verify the integration, run both applications at the same time:

Frontend → `http://localhost:5173`

Backend → <http://localhost:4000>

Because the Profile and Resume endpoints are protected, the frontend must send a valid JWT with its requests. For development testing, a valid token can temporarily be stored in the browser's `localStorage`:

```
localStorage.setItem('token', 'YOUR_JWT_TOKEN')
```

The React frontend can then retrieve the token and send it with API requests:

```
const token = localStorage.getItem('token');
```

```
fetch('http://localhost:4000/api/profile', {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
});
```

For the **Profile feature**, test the complete flow:

```
Profile page  
↓  
GET /api/profile  
↓  
Display saved profile  
↓  
Edit information  
↓  
PUT /api/profile  
↓  
SQLite updated  
↓  
Refresh page  
↓  
Updated information still appears
```

A good verification is to modify something obvious, such as the headline, save the profile, and refresh the page. If the new value remains after refreshing, the frontend is successfully sending data to the backend and retrieving the saved data from SQLite.

For **Resume Management**, open the Resume page and verify that existing resumes are retrieved using:

```
GET /api/resume
```

Then test uploading a new PDF or DOCX. The frontend should send a multipart request to:

```
POST /api/resume
```

After uploading, verify that the resume appears in the list with its filename and file size. Download and delete operations can also be tested to verify the remaining backend endpoints.

If information successfully reaches the frontend but displays incorrectly, such as an **Invalid Date**, investigate the frontend formatting before assuming that the backend request failed.